Pre Study

Literature review

Bruno Carchia
carchia@kth.se

April 29, 2026

1 Introduction

This section provides a critical analysis of the literature and technical reports that informed the selection of the methodologies and technologies employed in this research. Each reviewed paper contributed specifically to the architectural design or the strategic formulation of the Reinforcement Learning agent. To ensure clarity, each work is presented using the following structure:

- **Core Summary:** A brief overview of the research objectives and findings of the paper.
- **Technical Contribution:** An explanation of how the paper's insights directly influenced the development of this thesis.

2 An advanced learning environment and a scalable deep reinforcement learning approach for rolling stock circulation on urban rail transit line

This study [8] addresses the Rolling Stock Circulation (RSC) problem by formulating it as a multi-objective optimization task within a Deep Reinforcement Learning framework. The primary objectives identified are the minimization of fleet utilization, the balancing of maintenance workloads, and the stabilization of depot stock levels.

State Space Representation

The authors define a hierarchical state representation where the global state s^t is a concatenation of individual rolling stock statuses s_k^t. The local state for each asset k is defined as a six-dimensional feature vector:

$$s_k^t = \left(n_k^t, \bar{c}_k^t, \bar{b}_k^t, f_k^t, o_k^t, g_k^t\right)$$

This vector captures critical operational constraints, including cumulative trip counts (n_k^t), remaining service life ($\bar{c}_k^t$), and turnaround readiness (f_k^t). Notably, the inclusion

of operation direction (o_k^t) and a timetable pointer (g_k^t) ensures the agent maintains the Markov property by encoding the physical feasibility of future assignments.

Action Space

The action space $A = \{1, 2, \ldots, |K|\}$ is discrete, where each action corresponds to the selection of a specific physical asset from the fleet. To handle the "hard constraints" of rail operations (e.g., assigning a busy train), the authors utilize a penalty-based approach rather than strict action masking. This forces the agent to learn the underlying feasibility logic through the negative reinforcement of invalid selections.

Reward Mechanism

A dual-reward structure is employed to handle the credit assignment problem:

1. **Dense Step-wise Reward (r'):** Focused on immediate utilization efficiency and constraint satisfaction. It penalizes unnecessary depot extractions (r_a) and rewards the selection of idle assets with high turnaround times (r_v), promoting a "First-In, First-Out" logic.

2. **Sparse Terminal Reward (Z):** A weighted sum of normalized objectives calculated at the end of the operation horizon. This term targets fleet-wide fairness (workload standard deviation z_3) and depot equilibrium (z_2).

(a) Actor Network

(b) Critic Network

Figure 1: Architecture of the proposed Actor-Critic Neural Network. Taken from (Yang et al., 2025 [8])

Neural Network Architecture

The problem is solved using the Proximal Policy Optimization (PPO) algorithm. As illustrated in Figure 1, the Actor and Critic networks utilize a fully-connected Multi-Layer Perceptron (MLP) structure with hyperbolic tangent (tanh) activation functions.

Technical Contribution and Relevance to Thesis

This paper provides the foundational logic for my research, particularly in the structuring of sparse and dense rewards to balance local efficiency with global maintenance goals. However, while the authors utilize a global MLP that scales linearly with fleet size, my work improves upon this by implementing a **Shared-Weight MLP**. This ensures that the model is permutation-invariant and can generalize to varying fleet sizes without re-training—a significant limitation observed in the original paper's architecture.

Feature	Reference Paper	This Thesis
Algorithm	PPO (Actor-Critic)	PPO (Actor-Critic)
Action Space	Discrete (Fixed $\|K\|$)	Continuous Weights (Scalable)
NN Architecture	Global Flat MLP	Shared-Weight Architecture
Observation Space	Fixed Global Vector	Masked Relational Features
Maintenance Logic	Reactive (Fixed Thresholds)	Proactive (Weibull-based)
Simulation Type	Step-based	Event-driven

Table 1: Comparative analysis between the reference study and the proposed DRL approach.

3 Deep Reinforcement Learning for Integrated Train Scheduling and Rolling Stock Circulation

A significant contribution to integrated rail optimization is presented by Zhao et al. [9], who propose a Deep Reinforcement Learning (DRL) approach to solve the joint problem of train scheduling and rolling stock circulation. The authors utilize a Hybrid Proximal Policy Optimization (HPPO) algorithm to manage a hybrid action space characterized by both continuous scheduling decisions and discrete rolling stock assignments. The primary objective is to minimize a multi-objective cost function comprising passenger waiting time and operational expenses under complex constraints.

In this framework, the DRL agent acts as a dispatcher, sequentially determining section running times, station dwell times, and the allocation of rolling stock units—either by deploying units from the depot or reassigning units arriving from previous services.

State Space Representation

The state space S_n at step n captures the temporal and logistical dynamics of the network:

$$S_n = (d_{n,i}, a_{n,i}, l_{n,i}, o_{n,k}) \tag{1}$$

where $d_{n,i}$ and $a_{n,i}$ denote departure and arrival times at station i, $l_{n,i}$ represents the passenger queue length post-departure, and $o_{n,k}$ indicates the availability time of rolling stock unit k.

Action Space Representation

The action space A_n is hybrid, formulated as:

$$A_n = (h_{n+1}, r_{n+1,i}, e_{n+1,i}, f_{n+1,k}) \tag{2}$$

Here, the headway (h_{n+1}), section running times ($r_{n+1,i}$), and dwell times ($e_{n+1,i}$) are continuous variables, while the rolling stock assignment ($f_{n+1,k} \in \{0, 1\}$) is a discrete binary decision.

Reward Mechanism

The reward function is defined as the negative weighted sum of three components:

1. *Passenger Waiting Costs*: Calculated as the sum of waiting time for newly arrived passengers and those stranded due to capacity limits from previous services.

2. *Operational Costs*: Defined by the total cycle time for service n, calculated as $(a_{n,2I} - d_{n,I})$.

3. *Usage Costs*: Proportional to the total number of active rolling stock units $\sum_{k \in K} \eta_k$.

Neural Network Architecture

To handle the hybrid nature of the actions, the architecture employs a dual-actor system 2. A discrete actor (π_{θ_d}) outputs the probability distribution for rolling stock assignments, while a continuous actor (π_{θ_c}) parameters a multivariate Gaussian distribution to sample scheduling variables.

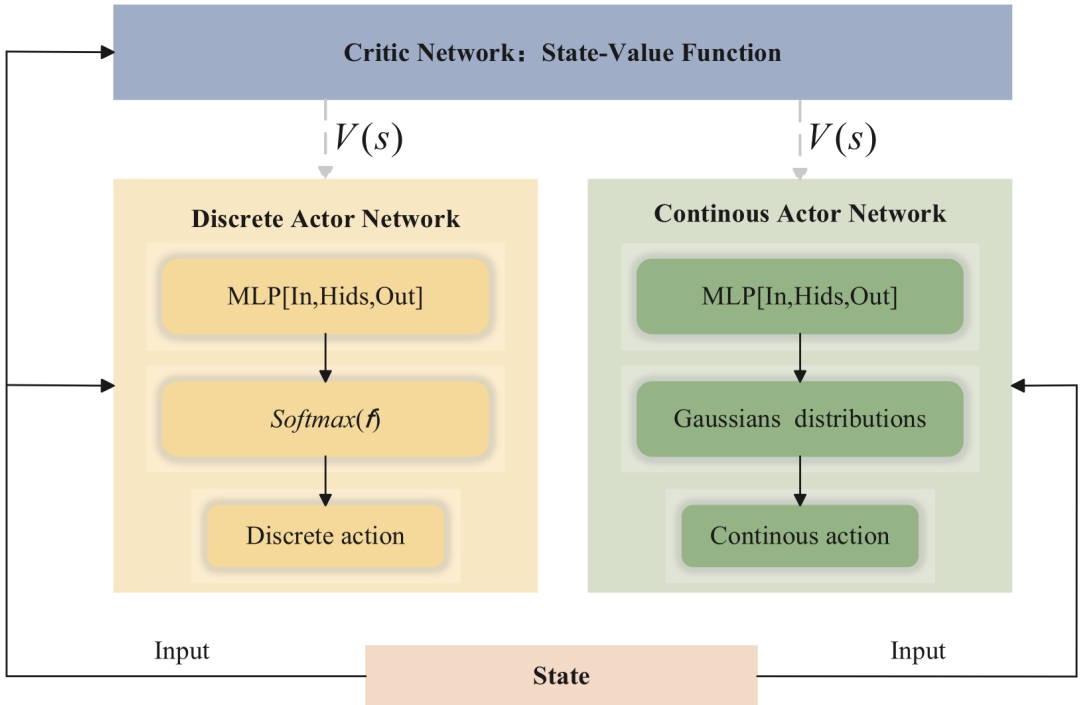

Figure 2: Architecture of the proposed Actor-Critic Neural Network. Taken from (Zhao et al., 2026 [9])

Relevance and Comparative Analysis

While the study by Zhao et al [9]. addresses a more geographically complex passenger network, it provides a vital methodological benchmark for this thesis. Both approaches utilize action masking to enforce operational constraints; however, several key distinctions exist. While Zhao et al. [9] optimize for passenger flows using a hybrid HPPO actor-critic model, this thesis focuses on freight logistics (specifically the transport of timber—within a two-line system).

Furthermore, instead of the dual-actor hybrid approach (HPPO) used in the cited study, this research employs a standard PPO algorithm with a single continuous actor. In this thesis, discrete requirements are handled by mapping continuous outputs to discrete values via a specialized mapping strategy, rather than maintaining a separate discrete actor network.

4 Solving high-speed rolling stock planning with an improved LSTM-PPO reinforcement learning algorithm

The work presented by Su et al. [7] investigates the Rolling Stock Planning (RSP) problem utilizing a PPO framework integrated with a Long Short-Term Memory (LSTM) network. To prevent the agent from converging on local optima, the authors incorporate a simulated annealing strategy, effectively balancing global exploration with local refinement. The choice of an LSTM architecture is specifically intended to capture temporal dependencies and provide the agent with a recursive memory of the episode's history.

A fundamental assumption of this study is the prioritization of a fixed maintenance threshold over train departures, ensuring safety constraints are satisfied. The operational environment is modeled with three stations and a fixed scheduling regime, establishing a scenario closely aligned with the parameters of this thesis.

State Space Representation

The state at time step t is defined by the history of assignments V_t and the current task x_t. The history V_t represents a comprehensive record of all train services already allocated to the v rolling stock units (RSUs), while x_t denotes the specific service currently awaiting assignment:

$$V_t = \left\{U_t^1, U_t^2, \ldots, U_t^v\right\} = \left\{\left\{u_1^1, u_2^1, \ldots, u_r^1\right\}, \ldots, \left\{u_1^v, u_2^v, \ldots, u_r^v\right\}\right\}$$

Action Space Representation

The agent is tasked with mapping the pending train service (x_t) to an available RSU (v_i). The resulting action is formulated as:

$$a_t = (v_i, x_t)$$

Consistent with the methodology adopted in this research, the action space is constrained by a filtering mechanism that automatically excludes actions violating operational requirements, such as maintenance schedules.

Reward Mechanism

The reward function is a multi-objective scalar derived from three primary components, each modulated by specific hyperparameters:

1. r_t^{connect}: A positive reward for establishing a feasible connection between successive train services on the same RSU.

2. r_t^{save}: An incentive aimed at minimizing the total number of RSUs required for operations.

3. r_t^{penalty}: A penalty term for the violation of hard constraints, primarily regarding fixed maintenance intervals.

Technical Contribution and Relevance to Thesis

The problem addressed by Su et al. [7] serves as a foundational benchmark for this thesis, although it represents a slightly simplified version of the real-world system as it focuses exclusively on RSU efficiency without considering passenger demand.

There is a significant alignment between this thesis and Su et al. [7] regarding the prioritization of system safety; both models enforce maintenance requirements as a higher priority than schedule adherence. However, the maintenance modeling differs in flexibility: while Su et al. [7] restricts maintenance to nighttime windows, this thesis allows the agent to execute early maintenance at any time to accommodate operational needs. Furthermore, while the present work omits dwell times as a deliberate simplification, this feature remains a viable candidate for future integration. Finally, although this thesis utilizes a Multi-Layer Perceptron (MLP) architecture rather than the LSTM employed by Su et al. [7], the algorithmic structure of their work provided significant inspiration for the development of my proposed solution.

Algorithm 1 LSTM-PPO for rolling stock planning.

Require: Initialized actor π_θ, critic V_ϕ, LSTM state h_0, learning rates α_θ, α_ϕ, clipping parameter ϵ, epochs K

Initialize rolling stock planning environment
for each training iteration **do**
  Collect episodes:
  for each circulation planning episode **do**
    Reset environment and initialize LSTM hidden state h_0
    for each train service step t **do**
      Observe current state s_t (e.g. RSU status, depot availability, task x_t)
      Compute action distribution $\pi_\theta(a_t \mid s_t, h_{t-1})$
      Sample assignment decision a_t
      Execute a_t, observe reward r_t and next state s_{t+1}
      Store (s_t, a_t, r_t, h_{t-1}) in buffer
      Update hidden state: $h_t \leftarrow \text{LSTM}(s_t, h_{t-1})$
    end for
  end for
  Estimate advantages $\hat{A}_t$ and returns $\hat{R}_t$ via GAE
  for K epochs **do**
    Sample mini-batches from buffer
    for each mini-batch **do**
      Compute probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t, h_{t-1})}{\pi_{\theta_{\text{old}}}(a_t \mid s_t, h_{t-1})}$$

      Compute clipped surrogate loss:

$$L^{\text{PPO}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$$

      Update actor parameters: $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta L^{\text{PPO}}$
      Update critic loss: $L_V = \|V_\phi(s_t, h_{t-1}) - \hat{R}_t\|^2$
      Update critic parameters: $\phi \leftarrow \phi - \alpha_\phi \nabla_\phi L_V$
    end for
  end for
end for

Figure 3: PPO-LSTM Algorithm. Taken from (SU et al., 2026 [7])

5 Rolling Stock Planning with Integrated Maintenance Constraints: A Rolling Horizon MILP Approach

The work by Bröchin et al. [1] presents a robust optimization framework for rolling stock circulation that explicitly integrates multi-tier maintenance requirements. Unlike traditional "maintenance-first" approaches, which often decouple service scheduling from technical service, this study utilizes a Mixed-Integer Linear Programming (MILP) model within a rolling horizon framework to optimize short-term operational efficiency alongside long-term fleet reliability.

Operational Complexity and Horizon Segmentation

The model accounts for high-fidelity operational constraints, including specific maintenance categories (T001, ESA, and DLR) and workshop availability fluctuations due to holidays or capacity limits. A central feature of the methodology is the temporal segmentation of the decision-making window into two distinct phases: the Activity Horizon and the Maintenance Horizon, as illustrated in Figure 4.

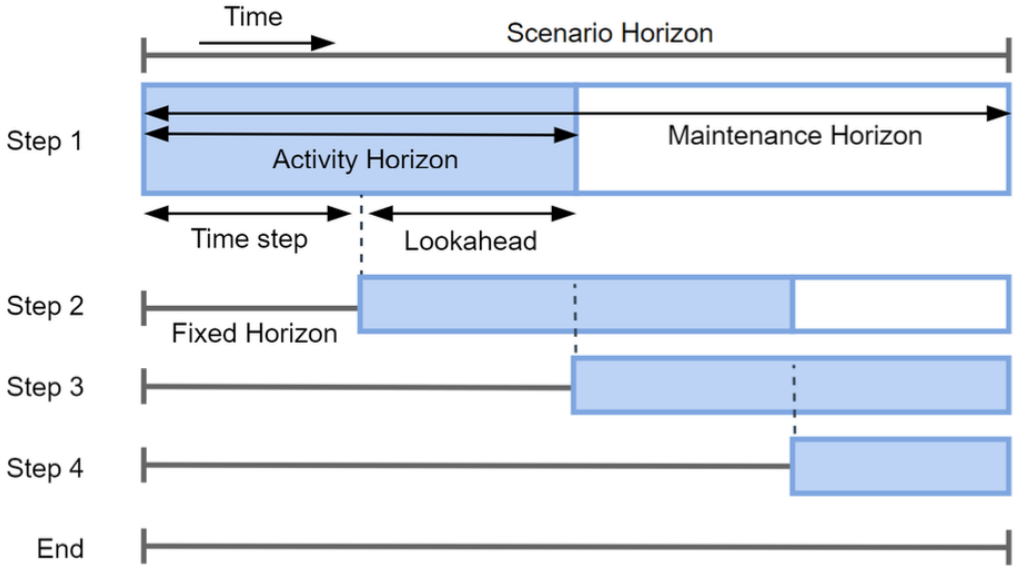

Figure 4: Temporal decomposition of the rolling horizon approach, distinguishing between detailed activity scheduling and expected maintenance requirements. Taken from Bröchin et al. (2023) [1].

By solving the MILP model at each step of the rolling horizon, the system performs a trade-off analysis between immediate revenue-generating activities and the expected future costs of maintenance neglect or suboptimal scheduling.

Multi-Layered Graph Architecture

The formulation utilizes a "layered flow" network architecture, which is critical for maintaining computational tractability. The graph is decomposed into:

- **Activity Flow Layer:** A directed graph $G_A = (V_A, A_A)$ where nodes represent service trips or maintenance slots, and arcs represent feasible transitions (turnarounds, repositioning, and coupling).
- **Maintenance Flow Layers:** Specialized layers for each maintenance type $t \in T$ that track the elapsed time/distance since the last service event, ensuring safety limits are not exceeded.

Objective Function

The optimization goal is to minimize a composite cost function representing both operational and technical expenditures:

$$\min \sum_{a \in A_A} c_a x_a + \sum_{t \in T} \sum_{a \in A_t} d_{a,t} w_{a,t} \tag{3}$$

This function penalizes deadheading (empty runs) and minimizes deviations from the original plan, effectively balancing economic performance with schedule stability.

Technical Contribution and Relevance to Thesis

The research by Bröchin et al. [1] is highly relevant as it demonstrates the necessity of integrating maintenance into the primary circulation plan to avoid "Maintenance Spikes." However, a significant limitation of their MILP-based approach is its computational latency, which makes it less suitable for the real-time, event-driven rescheduling required in dynamic freight environments. The papers is trying to resolve the following problem,: the assignment and reassignment of wagons to transport freight from origins to destinations within a railway network where there multiple stations and types of wagons. In contrast to their layered network flow, this thesis proposes an Actor-Critic DRL architecture. While Bröchin et al. [1] manage complexity through "commoditization" of flow, my approach utilizes a Shared-Weight MLP to achieve the same fleet-wide scalability without the need for solving expensive linear programs at every decision point. This allows for near-instantaneous rescheduling when stochastic events (such as the Weibull-distributed failures modeled in my work) occur.

6 Dynamic wagon assignment and reassignment for pick-up and delivery requests: A deep reinforcement learning framework to address uncertainty in locomotive availability

The research conducted by Habibi et al. (2026) [5] addresses the complexities of railway logistics through the application of Deep Reinforcement Learning (DRL), specifically utilizing the Proximal Policy Optimization (PPO) algorithm. The primary objective of their framework is the maximization of operational efficiency by ensuring the fulfillment of delivery requests while simultaneously minimizing logistical costs and temporal delays.

State Space Representation

The agent observes a discrete environment where the state s_t at any given time t is defined as:

$$s_t = (Bv^K_{t,m}, ya_{t,w,i})$$

This representation captures two critical components:

- The current inventory levels of freight across all pending requests.
- The distribution and availability of specific wagon types across the network's stations.

This state space design mirrors the requirements of complex railway environments, providing the necessary visibility into resource allocation and demand fulfillment.

Action Space Representation

A recurring challenge in railway scheduling—as observed in the model developed for this thesis—is the lack of scalability when the number of stations or assets increases. To mitigate the "curse of dimensionality" inherent in large-scale discrete action spaces, Habibi et al. introduce a set of twelve heuristic rules 5. These rules act as an abstraction layer for the agent, constraining the action space to high-level wagon assignment and re-assignment decisions. This approach contrasts with the raw weight-based selection mechanism utilized in my model, where the agent directly outputs weights for locomotive selection. While my model currently grants the agent greater freedom, the heuristic approach suggests a viable pathway for reducing complexity in future iterations of the project.

Proposed heuristic rules for wagon assignment

Assignment of wagons from the origin stations of requests based on specific assignment conditions and, in stations with the same origin request, selecting the request under selection priority criteria

Symbols	Priority criterion
$RU_{1,1}$	Lower inventory of freight
$RU_{1,2}$	Higher inventory of freight
$RU_{1,3}$	Higher commitment completion
$RU_{1,4}$	Lower commitment completion
$RU_{1,5}$	No assignment in this period

Proposed heuristic rules for wagon reassignment

Reassignment of wagons from each station to the origin of a request, while the station was the destination of that request or is within a specified distance from its origin, and ensuring that the type of wagons being reassigned is suitable for the freight of that request. When multiple requests satisfy these conditions, the request is chosen according to the following priority criteria:

Symbols	Priority criterion
$RU_{2,6}$	Higher inventory of freight at origin.
$RU_{2,7}$	Lower inventory of freight at origin
$RU_{2,8}$	Higher commitment completion
$RU_{2,9}$	Lower commitment completion
$RU_{2,10}$	Lower number of available wagons at origin
$RU_{2,11}$	Higher number of available wagons at its origin is prioritized.
$RU_{2,12}$	No reassignment in this period.

Figure 5: Heuristic rules for wagon assignment or re-assignment. Taken from Habibi et al. (2026) [5].

Reward Mechanism

The reward function is formulated to align the agent's behavior with corporate profitability. It is composed of four distinct weighted components:

1. Positive reinforcement for revenue generated from fulfilled freight deliveries.
2. Penalties associated with delivery delays (lateness).
3. Operational costs associated with the transportation of fully loaded wagons.
4. Variable costs incurred by the movement of empty wagons (positioning costs).

Neural Network Architecture

The actor-critic framework employs three fully connected hidden layers, each consisting of 256 neurons. Rectified Linear Unit (ReLU) functions are utilized for non-linear activation. The optimization process is conducted via the Adam optimizer, utilizing a

learning rate of 0.002 and a discount factor (γ) of 0.9. These parameters served as the baseline configuration for my model before empirical fine-tuning was performed to adapt to the specific requirements of locomotive selection.

Technical Contribution and Thesis Relevance

The work by Habibi et al. served as a foundational benchmark for this research. While my model focuses on a multi-locomotive selection environment built with the Gymnasium framework, the architectural parallels in the PPO implementation provided a robust starting point for determining network size and hyperparameters.

Critically, however, the Habibi model assumes a deterministic environment. A significant gap identified during this review is the absence of modeling for stochastic failures or external uncertainty—events that are ubiquitous in real-world railway operations. While my model improves upon the user-interface and visual action-tracking, the next phase of this thesis aims to integrate the heuristic-based dimensionality reduction seen in Habibi's work while introducing environmental uncertainty to better simulate the "real-world" drift between planned schedules and execution.

7 An actor-critic deep reinforcement learning approach for metro train scheduling with rolling stock circulation under stochastic demand

The study by Ying et al. [6] investigates an actor-critic Deep Reinforcement Learning (DRL) framework for metro train scheduling, with a specific focus on the rotation of a constrained fleet of rolling stock units. The system is modeled as a closed-loop network of stations $\mathbb{I} = \{1, 2, \ldots, I\}$, where I_0 represents the depot for train dispatching. The objective is to assign a fixed fleet of K trains to N service runs over a predefined horizon, ensuring efficient circulation under capacity constraints.

State Space Representation

The state tuple s_n at each stage n encapsulates the observed environment for the agent:

$$s_n = (\tau_{n,i}, \sigma_{n,i}, l_{n,i}) \tag{4}$$

where $\tau_{n,i}$ and $\sigma_{n,i}$ denote the arrival and departure times of service n at station i, respectively. The term $l_{n,i}$ represents the passenger accumulation remaining at station i following the departure of service n. While structurally similar to the work of Zhao et al. [9], this formulation places greater emphasis on the specific requirements of rolling stock planning.

Action Space Representation

The action vector x_n determined at stage n for the subsequent service $n+1$ is defined as:

$$x_n = (h_{n+1}, u_{n+1,i}, w_{n+1,i}) \tag{5}$$

where h_{n+1} is the headway between service n and $n+1$ at the dispatching point, $u_{n+1,i}$ is the section running time between station pairs $(i, i+1)$, and $w_{n+1,i}$ denotes the station dwell times. These parameters are subject to bound-clipping to maintain operational

feasibility. A critical assumption in this model is that the fleet size K is smaller than the total service requirements, thereby necessitating an effective circulation and reuse strategy.

Reward Mechanism

The reward function is a weighted multi-objective scalar derived from two perspectives:

1. Passenger Utility: A cost function $c_n^W = \zeta^W \sum_{i=1}^{I} c_{n,i}^W$ quantifies the waiting time for passengers. The factor ζ^W serves to monetize total waiting time based on established economic standards [3, 2].

2. Operator Efficiency: The operational cost c_n^V accounts for energy consumption and maintenance-related expenditures:

$$c_n^V = \sum_{i=1}^{I} \left(\zeta^F F(v_{n,i}) d_{i,i+1} + \zeta^T u_{n,i} + \zeta^D d_{i,i+1} \right) \tag{6}$$

   where $F(v_{n,i})$ represents energy consumption as a function of speed, and $\zeta^F, \zeta^T, \zeta^D$ are unit cost factors for fuel, time-related expenditures, and distance-related wear, respectively.

Neural Network Architecture

To mitigate the curse of dimensionality, the authors implement the Deep Deterministic Policy Gradient (DDPG) algorithm. The architecture employs two-layer artificial neural networks for both the actor and the critic, utilizing non-linear activation functions and the Adam optimizer [4]. A linear mapping function f is used to scale the bounded action space $[-1, 1]$ to real-world operational values.

Technical Contribution and Thesis Relevance

This paper serves as a primary reference for the implementation of action-value clipping and linear mapping within this thesis. Furthermore, the inclusion of multi-faceted maintenance costs in the reward function provided a methodological foundation for our model, despite our simplified parameter set.

Most significantly, Ying et al. highlight the inherent difficulty of achieving convergence in environments characterized by stochasticity and random failure events. This aligns with the findings of the present research: while the system demonstrates high operational performance, the training curves exhibit the characteristic volatility (as seen in Figure 6) common in high-uncertainty DRL applications. This research confirms that while reducing stochastic variables leads to smoother convergence, it results in a less robust model capable only of finding trivial solutions.

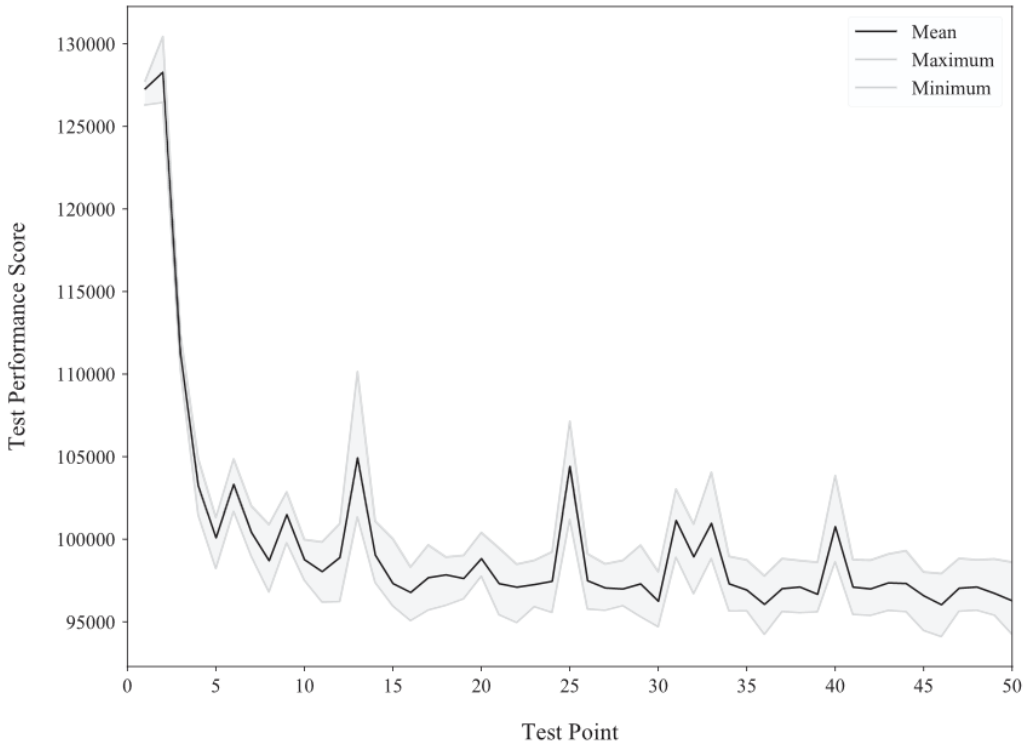

Figure 6: Test of the actor-critic agent during the training process.Taken from (Ying et al., 2020 [6])

References

[1] Manuel Bröchin, Reto Ramseier, and Kaspar Schüpbach. Rolling stock planning with maintenance constraints by a rolling horizon approach. 04 2023.

[2] A. H. F. Chow and A. Pavlides. Cost functions and multi-objective timetabling of mixed train services. *Transportation Research Part A*, 113:335–356, 2018.

[3] Department for Transport (DfT). Tag unit 3.5.6 values of time and vehicle operating costs, transport analysis guidance data book. Technical report, Department for Transport, United Kingdom, 2014.

[4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.

[5] Parivash Habibi, Mohammad Tamannaei, and Hamid Zarei. Dynamic wagon assignment and reassignment for pick-up and delivery requests: A deep reinforcement learning framework to address uncertainty in locomotive availability. *Applied Soft Computing*, 187:114351, 2026.

[6] Cheng shuo Ying, Andy H.F. Chow, and Kwai-Sang Chin. An actor-critic deep reinforcement learning approach for metro train scheduling with rolling stock circulation under stochastic demand. *Transportation Research Part B: Methodological*, 140:210–235, 2020.

[7] Xiangyu Su, Yixiang Yue, Bin Guo, Zanyang Cui, and Boyan Li. Solving high-speed rolling stock planning with an improved lstm-ppo reinforcement learning algorithm. *Transportmetrica A Transport Science*, 2026.

[8] Yuhua Yang, Haoyang Huo, Nikola Bešinović, Yichen Sun, and Shaoquan Ni. An advanced learning environment and a scalable deep reinforcement learning approach for rolling stock circulation on urban rail transit line. *Transportation Research Part C: Emerging Technologies*, 171:104976, 2025.

[9] Xiaoli Zhao, Dewei Li, and Xinyu Bao. A deep reinforcement learning approach for integrated optimization of train scheduling and rolling stock circulation planning. *Computers & Industrial Engineering*, 213:111784, 2026.